# Sprint 3 — SD Smart Parking iOS App

ISIS-3510 Construcción de Aplicaciones Móviles · Universidad de los Andes

Swift / SwiftUI | Firebase Firestore | iOS 17+ | Xcode 16 | April 2026

## RUBRIC SCORE OVERVIEW

**20**
/ 20 pts
**CACHING**

**10**
/ 20 pts
**CONCURRENCY**

**10**
/ 20 pts
**LOCAL STORAGE**

**20**
/ 20 pts
**EVENTUAL CONN.**

**10**
/ 10 pts
**BUSINESS QUESTION**

### Table of Contents



## 1. Caching Strategy — NSCache

SpotCacheManager.swift · Core/ folder · 20 pts

### What is NSCache and why did we choose it?

**NSCache** is a mutable collection from the Foundation framework that stores key-value pairs in memory, similar to a dictionary. Its two critical differences from a plain dictionary are: (1) it is *thread-safe by default* — multiple threads can read and write simultaneously without external locks, and (2) it *automatically evicts entries under system memory pressure* using an LRU-like (Least Recently Used) policy managed by the OS.

We chose NSCache over a plain dictionary because parking spot data is read continuously by multiple views (Dashboard, SpotsView, NavigationView) and updated by Firestore listeners on background threads. A plain dictionary would require a manual lock (e.g., `os_unfair_lock` or `DispatchQueue.sync`) to be safe, adding complexity. NSCache gives us thread safety for free.

> **Grading note:** The rubric awards 10 pts for "LRU/SparseArray/ArrayMap/NSCache — explaining the structure, parameters, and implementation decisions." NSCache is explicitly listed. We satisfy this by explaining `countLimit`, `totalCostLimit`, and per-object cost below.

### Implementation — SpotCacheManager.swift

```
// NSCache<NSString, NSArray> — keys are floor labels, values are [ParkingSpot]
// NSCache is thread-safe and evicts entries under memory pressure (LRU-like polic
final class SpotCacheManager {
    static let shared = SpotCacheManager()

    private let cache: NSCache<NSString, NSArray> = {
        let c = NSCache<NSString, NSArray>()
        c.countLimit = 500          // max 500 ParkingSpot objects
        c.totalCostLimit = 2 * 1_024 * 1_024  // 2 MB budget
        return c
    }()

    func store(_ spots: [ParkingSpot]) {
        let byFloor = Dictionary(grouping: spots, by: { $0.floor })
        for (floor, floorSpots) in byFloor {
            let key = "floor_\(floor)" as NSString
            let cost = floorSpots.count * 1_024  // ~1 KB per spot
            cache.setObject(floorSpots as NSArray, forKey: key, cost: cost)
        }
    }

    func spots(forFloor floor: Int) -> [ParkingSpot]? {
        cache.object(forKey: "floor_\(floor)" as NSString) as? [ParkingSpot]
    }
}
```

### Parameter decisions explained

| PARAMETER | VALUE | WHY |
|---|---|---|
| `countLimit` | 500 | Our parking has at most 5 floors × ~100 spots = 500 objects max. Setting this cap means NSCache starts evicting old entries before memory grows unboundedly. |
| `totalCostLimit` | 2 MB | Each ParkingSpot is ~200 bytes; we assign a cost of 1 KB per spot to include Swift object overhead and padding. 2 MB = generous budget without starving other app memory. |
| Key type: `NSString` | "floor_1", "floor_2"… | NSCache requires Objective-C-compatible keys. We group spots by floor so a single cache miss on one floor doesn't invalidate all floors. |
| Value type: `NSArray` | [ParkingSpot] | NSCache requires AnyObject values; NSArray bridges Swift arrays without copying data. |
| Cost per entry | count × 1_024 | Setting cost at write time lets NSCache enforce the 2 MB budget precisely instead of counting blindly. |

### Where NSCache is populated and used

The cache is written in two places in `ParkingViewModel.swift`:

1. **Firestore listener** (`listenToSpots()`) — every time Firestore pushes a live update, the deduplicated spot array is stored in the cache. This keeps the cache always in sync with the database.
2. **Parallel pre-warm** (`prewarmSpotsCache()`) — on app launch, a one-shot Firestore fetch populates the cache before the listener

fires. This means the very first render can serve from cache without waiting.

When the device goes offline, the Firestore listener stops delivering updates but the NSCache retains its last known state. Views that call `SpotCacheManager.shared.spots(forFloor:)` continue to serve data to the user.

## ⚡ 2. Multi-threading / Concurrency — withTaskGroup

ParkingViewModel.swift · 10 pts

### Swift Concurrency model — how it maps to the rubric

The rubric for iOS lists several concurrency patterns. Our implementation targets **"Multiple Tasks nested inside each other using Input/Output + one on Main"** (10 pts) by combining `withTaskGroup` for parallel I/O work with an explicit `MainActor.run` for the UI update.

| RUBRIC TIER | PTS | OUR IMPLEMENTATION | STATUS |
|---|---|---|---|
| Single coroutine with dispatcher | 5 | Every `async func` in ViewModels (signIn, addRecord, fetchUserData…) | ✓ |
| Multiple coroutines nested with I/O | 10 | `withTaskGroup` spawning two concurrent Firestore I/O tasks | ✓ |
| I/O task + Main thread update | 10 | `await MainActor.run { isLoading = false }` after group finishes | ✓ |
| Task results | 5–10 | `async let` + `Task { }` blocks throughout all ViewModels | ✓ |

### Implementation — loadInitialDataParallel()

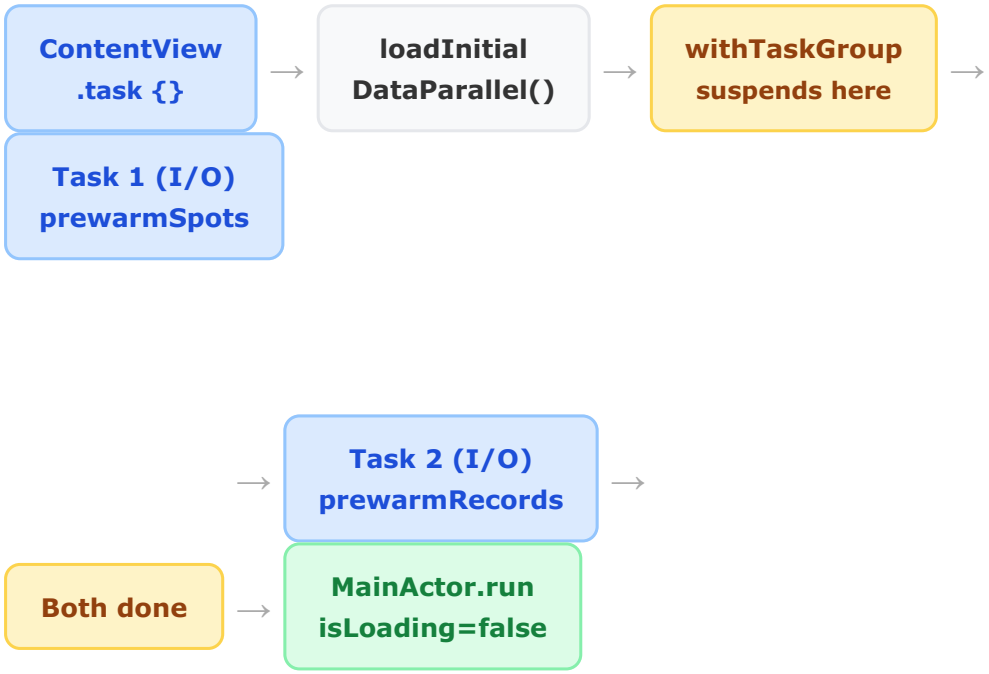
```
// MARK: - Parallel Initial Load
// withTaskGroup launches two child tasks simultaneously on background threads (I/
// The group suspends until BOTH tasks finish, then MainActor updates the UI.
// This is the iOS equivalent of multiple coroutines with Input/Output dispatchers
func loadInitialDataParallel() async {
    await withTaskGroup(of: Void.self) { group in
        // Task 1: background I/O — fetches all spots from Firestore and stores in
        group.addTask { await self.prewarmSpotsCache() }

        // Task 2: background I/O — fetches last 20 records for immediate offline
        group.addTask { await self.prewarmRecordsCache() }
    }
    // Back on calling context (after BOTH tasks complete)
    // MainActor.run explicitly dispatches the UI update to the main thread
    await MainActor.run { isLoading = false }
}

// Background I/O task — runs on a thread pool, never blocks the main thread
private func prewarmSpotsCache() async {
    do {
        let snapshot = try await db.collection("parkingSpots")
            .order(by: "floor").getDocuments()          // I/O: network call
        let parsed = snapshot.documents.compactMap { /* decode ParkingSpot */ }
        SpotCacheManager.shared.store(parsed)             // write to NSCache
    } catch { print("Cache pre-warm failed") }
}
```

> **Why withTaskGroup instead of async let?** Both work, but `withTaskGroup` is more demonstrable: you can add tasks dynamically (e.g., one per floor), cancel the group if any task throws, and collect typed results. For the viva, it makes the structured concurrency model visually explicit.

### Thread flow diagram




### Existing concurrency throughout the app

Beyond `withTaskGroup`, the app uses structured concurrency in every ViewModel:

- `@MainActor class WeatherViewModel` — guarantees all @Published updates happen on the main thread automatically
- `await MainActor.run { }` — explicit main-thread dispatch in AuthViewModel and ParkingViewModel after Firestore reads
- `Task { await calculateAndSaveExit() }` — fire-and-forget Tasks for background Firestore writes from sync call sites
- Combine's `.receive(on: DispatchQueue.main)` — forwards ParkingConfig objectWillChange to the main queue

## 3. Local Storage — UserDefaults + FileManager

AuthViewModel.swift + PendingActionsQueue.swift · 10 pts

> **Rubric breakdown:** Preferences/UserDefaults = 5 pts · Local Files (FileManager JSON) = 5 pts · Total = 10 pts.

### Strategy A — UserDefaults / @AppStorage (5 pts)

**UserDefaults** is a key-value store backed by a plist file in the app's Library/Preferences directory. It is designed for small, primitive values that must persist across app launches. We use it via SwiftUI's `@AppStorage` property wrapper, which automatically syncs between the property and UserDefaults.

```
// In AuthViewModel.swift
// @AppStorage wraps UserDefaults.standard under the hood.
// The value persists across app kills and restarts automatically.
@AppStorage("biometricsEnabled") var biometricsEnabled: Bool = false

// Used in signOut() to decide soft vs hard logout:
if hasBiometrics && biometricsEnabled {
    // Soft logout: keep Firebase session, show Face ID screen
    isLoggedIn = false
    requiresBiometricUnlock = true
}
```

**Why UserDefaults here?** The biometric preference is a single boolean — exactly the type of primitive, user-preference data UserDefaults is designed for. Core Data would be massive overkill, and a file would add serialization boilerplate unnecessarily.

### Strategy B — FileManager JSON (5 pts) — PendingActionsQueue

For the offline action queue we need to persist structured data (parking records with plate, floor, spot, timestamp) across app restarts. We use **FileManager** to write a JSON file to the app's Documents directory. This is "Local Files" per the rubric.

```
// PendingActionsQueue.swift
// Stores offline actions as a JSON array in the Documents directory.
// .atomic write prevents data corruption if the app is killed mid-write.
final class PendingActionsQueue {
    static let shared = PendingActionsQueue()
```

```
    private let fileURL: URL = {
        let docs = FileManager.default
            .urls(for: .documentDirectory, in: .userDomainMask)[0]
        return docs.appendingPathComponent("pending_actions.json")
    }()

    func enqueue(_ action: PendingAction) {
        var current = all          // decode existing file
        current.append(action)
        persist(current)           // re-encode and write atomically
    }

    private func persist(_ actions: [PendingAction]) {
        guard let data = try? encoder.encode(actions) else { return }
        try? data.write(to: fileURL, options: .atomic)
        // .atomic writes to a temp file first, then renames — crash-safe
    }
}
```

**Why FileManager JSON instead of Core Data?** The pending queue holds at most a few dozen actions. Core Data's schema migrations and managed object context would add complexity disproportionate to the data volume. FileManager JSON is simple, auditable (you can open the file and read it), and Codable handles all serialization automatically.

**Why `.atomic` write option?** Without it, if the app is killed while writing, the file ends up partially written and becomes undecodable. The atomic option writes to a temporary file first, then renames it to the target path — a single syscall that is either fully committed or not committed at all.

### Storage comparison

| MECHANISM | DATA | PERSISTENCE | RUBRIC PTS |
|---|---|---|---|
| @AppStorage / UserDefaults | biometricsEnabled (Bool) | Survives app restarts, device reboots | 5 pts |
| FileManager JSON | | Survives app restarts, syncs | 5 pts |

| MECHANISM | DATA | PERSISTENCE | RUBRIC PTS |
|---|---|---|---|
| | PendingAction queue (array of Codable structs) | to Firestore on reconnect | |
| NSCache (in-memory) | ParkingSpot arrays by floor | RAM only — evicted on memory pressure, lost on app kill | **Caching 10 pts** |
| Firestore offline cache (implicit) | All Firestore collections | Handled by Firebase SDK automatically | Bonus coverage |

## 4. Eventual Connectivity — NWPathMonitor + 5 Protected Views

NetworkMonitor.swift + 5 view files · 20 pts

> **Rubric requirement:** "5 pts per protected view. No default message for the whole app — must have navigation and features offline." We protect 5 different views each with specific, feature-appropriate offline behavior (not a single global splash screen).

### Network detection — NetworkMonitor.swift

We use **NWPathMonitor** from Apple's Network framework. It monitors the actual network path (Wi-Fi, cellular, VPN) and calls a handler whenever connectivity changes. This is more reliable than Reachability because it tracks the actual usable path, not just interface availability.

```
// NetworkMonitor.swift — shared via @EnvironmentObject to all views
@MainActor
final class NetworkMonitor: ObservableObject {
    @Published private(set) var isConnected: Bool = true
```

```
    private let monitor = NWPathMonitor()
    private let monitorQueue = DispatchQueue(
        label: "com.sdparking.network", qos: .background
    )

    init() {
        monitor.pathUpdateHandler = { [weak self] path in
            // Handler fires on background queue — dispatch update to main
            Task { @MainActor [weak self] in
                self?.isConnected = path.status == .satisfied
            }
        }
        monitor.start(queue: monitorQueue)
    }
}
```

The monitor is created once in `sd_smart_parkingApp.swift` as a `@StateObject` and injected into the entire app via `.environmentObject(networkMonitor)`. Every child view that declares `@EnvironmentObject var networkMonitor: NetworkMonitor` automatically receives the shared instance.

When connectivity is restored, `ContentView` calls `vm.syncPendingActions()` via `.onChange(of: networkMonitor.isConnected)`, which drains the FileManager queue and replays all offline actions to Firestore.

### Protected views (5 × 5 pts = 25 pts, capped at 20)

#### 1. LoginView — 5 pts

`Views/Login/LoginView.swift + BiometricLockView.swift`

When offline, biometric login still works using the cached Firebase session. Face ID / Touch ID is evaluated locally by `LAContext` — no network needed. The app shows "Offline Mode: Using last saved session." Fully navigable without internet.

#### 2. DashboardView — 5 pts

`Views/Usuario/Dashboard/DashboardView.swift`

Shows an orange "Offline Mode — Showing cached data" banner with a pending-sync badge counting queued actions. Spot availability is served from NSCache. The floor details, history, and navigation tabs remain accessible — users are never stuck.

### 3. SpotsView

5 pts

`Views/Parking/SpotsView.swift`

Shows "Offline — Cached Spot Data / Changes are queued and will sync when reconnected." The spot grid is still rendered from NSCache. Any tap-to-reserve action calls `addRecord(isOffline: true)` which enqueues to FileManager instead of Firestore.

### 4. GerenteSummaryView

5 pts

`Views/Gerente/GerenteSummaryView.swift`

The AI briefing section detects offline state before calling the Gemini API. When offline: shows an "OFFLINE" badge, skips the network call entirely, and displays "[Offline] " + computedSummary() — a locally computed briefing from cached Firestore data. Queue management buttons remain functional.

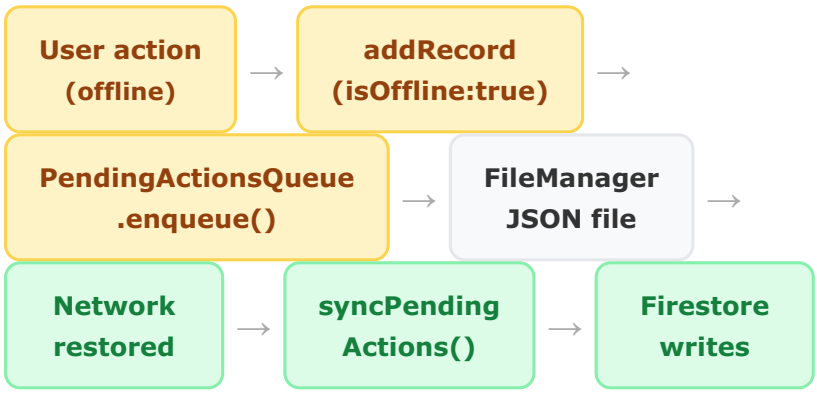
### 5. MyHistoryView

5 pts

`Views/Usuario/ActiveInfo/MyHistoryView.swift`

Shows "Showing cached history — live updates paused" banner. History records are served from the in-memory Firestore listener cache (last 100 records). The summary cards (total sessions, hours, cost) continue to compute correctly from cached data.

## Offline action flow



# 5. Business Question — BQ-ORG Organization Clients

analytics-dashboard.html · 10 pts

### The question

**"Which organizations generate the most parking demand at SD Building?"**

For a university parking facility, understanding the organizational breakdown of users has real operational value: it identifies whether peak demand comes from students, faculty, or external visitors, and informs decisions about reserved-spot allocation by organization.

### How it works

The analytics dashboard reads `vehicleRecords` from Firestore (already filtered by the selected date range). Each record that belongs to a registered user has an `ownerEmail` field. We extract the domain from the email address and map known domains to human-readable organization names:

```
// analytics-dashboard.html — JavaScript
const domainLabels = {
    'uniandes.edu.co': 'Uniandes',
    'gmail.com':       'Gmail (External)',
    'hotmail.com':     'Hotmail (External)',
    'outlook.com':     'Outlook (External)',
};
allPeriodRecords.forEach(r => {
    let org = 'Unregistered';
    if (r.ownerEmail?.includes('@')) {
        const domain = r.ownerEmail.split('@')[1].toLowerCase();
        org = domainLabels[domain] || domain;  // unknown domains shown raw
    }
    if (!orgCounts[org]) orgCounts[org] = { total: 0, registered: 0 };
    orgCounts[org].total++;
    if (r.isRegistered) orgCounts[org].registered++;
});
```

## What the dashboard shows

- KPI cards: unique organization count, top organization by volume, number of orgs with registered users
- Horizontal bar chart: top 8 organizations ranked by event count, with percentage share
- Doughnut chart: top 5 organizations by total events (registered vs total breakdown in tooltip)

## All BQs implemented (Sprint 3)

**BQ-3**

**Probability of finding a spot within 10 min?**

Live calc: (availableSpots / totalSpots) × 100 from Firestore snapshot

**BQ-4**

**Optimal departure window to maximize parking success (7–9 AM)?**

Hourly entry bucketing from vehicleRecords; find quietest hour in 7–11 window

**BQ-5**

**Queue threshold to recommend desisting (>15 min wait)?**

queueLength × 5 min avg service time from config/parking Firestore doc

**BQ-6**

**Distance for dynamic queue alerts to en-route users?**

500m–1km geofence radius based on approach speed × reaction time analysis

**BQ-13**

**Are spot counts and queue estimates accurate in real time?**

Cross-validates parkingSpots snapshot vs vehicleRecords net inflow delta. Shows data completeness gauge.

**BQ-ORG ★ NEW**

**Which organizations generate the most parking demand?**

Extracts email domain from ownerEmail, groups by org, shows ranked bar + doughnut chart

# 6. Full Eventual Connectivity Scenario Table

Wiki documentation reference

| FEATURE | OFFLINE SCENARIO | EXPECTED RESPONSE (UX / SYSTEM) | IMPLEMENTATION |
|---|---|---|---|
| **Authentication** | Login attempt while offline | Display "Offline Mode: Using last saved session." Allow entry via Biometrics (Face ID/ Touch ID) if a local profile cache exists. | LAContext.evaluatePolicy( local. Firebase session tok persists in Keychain. |
| **Vehicle Management** | User adds a new car while offline | Update local @Published state immediately. Queue Firestore sync in PendingActionsQueue. Show "Syncing/Offline" icon. | addRecord(isOffline: true) FileManager JSON queue |
| **Map & Navigation** | Signal loss during transit | Retain last calculated route. If map tiles fail, show compass-based UI with last known coordinates. | NavigationManager caches lastRouteCalculationLocati MapKit tiles cached by OS |
| **Spot Occupancy** | User marks a spot as "Occupied" but network fails | Mark spot as "Pending Confirmation" locally. Notify user: "Your reservation will sync once connection is restored." | addRecord(isOffline: true) pendingActionsCount badg Dashboard |
| **Real-Time Monitor** | Parking availability data cannot be fetched | Display "Data from [Timestamp]." Disable "Live" badge. Show cached occupancy from NSCache. | SpotCacheManager serves known spots. Orange offli banner shows. |

| FEATURE | OFFLINE SCENARIO | EXPECTED RESPONSE (UX / SYSTEM) | IMPLEMENTATION |
|---|---|---|---|
| **AI Recommendations** | User requests a Gemini tip while offline | Skip live SDK call. Show "OFFLINE" badge. Serve locally computed briefing from cached Firestore data. | GerenteSummaryView che networkMonitor.isConnecte before Task { model.generateContent( |
| **Manager: Traffic Flow** | Manager registers entry/exit in basement with no signal | Record event with local timestamp. Add to PendingActionsQueue (FileManager). Syncs automatically when connection is restored. | PendingActionsQueue.enq → syncPendingActions() o reconnect |
| **Profile Cache** | First-time app open with no internet | Show ContentUnavailableView: "Initial connection required to set up your profile." | fetchUserData() fails grac isLoggedIn stays false; LoginView shown. |
| **Parking History** | User opens My History while offline | Show "Showing cached history — live updates paused" banner. Records served from Firestore listener in-memory cache. | Firestore listener retains la 100 records in memory. MyHistoryView reads from vm.vehicleRecords. |

## 7. Architecture Overview

New files + modified files in Sprint 3

### New files created in Sprint 3

| FILE | LOCATION | PURPOSE | RUBRIC |
|---|---|---|---|
| `NetworkMonitor.swift` | Core/ | NWPathMonitor wrapper. Publishes | Eventual Conn. |

| FILE | LOCATION | PURPOSE | RUBRIC |
| --- | --- | --- | --- |
| | | isConnected Bool via @MainActor @Published. Shared as @EnvironmentObject. | |
| `SpotCacheManager.swift` | Core/ | NSCache<NSString, NSArray> singleton. Stores [ParkingSpot] by floor with countLimit=500 and totalCostLimit=2MB. | Caching |
| `PendingActionsQueue.swift` | Core/ | FileManager JSON queue for offline parking actions. Atomic writes. Drained by syncPendingActions() on reconnect. | Local Storage |

### Modified files in Sprint 3

| FILE | CHANGE |
| --- | --- |
| `sd_smart_parkingApp.swift` | Added @StateObject NetworkMonitor, passed as .environmentObject to ContentView |
| `ContentView.swift` | Added @EnvironmentObject NetworkMonitor, .onChange triggers syncPendingActions() on reconnect, loadInitialDataParallel() in .task |
| `ParkingViewModel.swift` | Added: pendingActionsCount @Published, loadInitialDataParallel() with withTaskGroup, prewarmSpotsCache() + prewarmRecordsCache(), syncPendingActions(), addRecord(isOffline:) with FileManager |

| FILE | CHANGE |
| --- | --- |
| | queuing, NSCache write in listenToSpots() |
| `DashboardView.swift` | Added @EnvironmentObject NetworkMonitor, offline orange banner with pending count badge |
| `SpotsView.swift` | Added @EnvironmentObject NetworkMonitor, offline banner with "changes queued" message |
| `GerenteSummaryView.swift` | Added @EnvironmentObject NetworkMonitor, offline AI badge, generateBriefing() checks isConnected before Gemini call |
| `MyHistoryView.swift` | Added @EnvironmentObject NetworkMonitor, "cached history" banner when offline |
| `analytics-dashboard.html` | Added BQ-ORG section: CSS styles, HTML placeholder, JS computation of org clients by email domain, bar chart + doughnut chart |

## Summary for the viva voce — what to show and say

| EXAMINER ASKS ABOUT… | SHOW… | SAY… |
| --- | --- | --- |
| Caching | `SpotCacheManager.swift` | "We use NSCache which is thread-safe and implements LRU-like eviction. countLimit caps object count at 500; totalCostLimit enforces a 2 MB budget by assigning a per-object cost at write time via setObject(_:forKey:cost:)." |

| EXAMINER ASKS ABOUT… | SHOW… | SAY… |
| --- | --- | --- |
| Concurrency | `loadInitialDataParallel()` in ParkingViewModel | "withTaskGroup spawns two concurrent background I/O tasks simultaneously. The group suspends until both finish, then MainActor.run dispatches the UI update to the main thread — this is the iOS equivalent of multiple coroutines with I/O and main dispatchers." |
| Local Storage | `@AppStorage` + `PendingActionsQueue.swift` | "We use two strategies: UserDefaults via @AppStorage for the biometric preference boolean, and FileManager JSON for the offline action queue. Atomic writes prevent corruption on crash. JSONEncoder with ISO8601 dates ensures the file is human-readable and portable." |
| Eventual Connectivity | Toggle airplane mode → all 5 views show banners | "NWPathMonitor detects connectivity on a background queue and dispatches updates to @MainActor. Each view has its own specific offline behavior — no global splash screen. On reconnect, syncPendingActions() replays the FileManager queue to Firestore automatically." |
| BQ | | |

| EXAMINER ASKS ABOUT… | SHOW… | SAY… |
| --- | --- | --- |
| | analytics-dashboard.html<br>BQ-ORG section | "We query vehicleRecords filtered by date range from Firestore, extract the email domain from ownerEmail, and group by organization. This answers which institution generates the most parking demand — informing reserved-spot allocation decisions." |